# Cvičenie: Gradientné metódy

Optimalizácia a optimálne riadenie

## Problém:

Chceme minimalizovať nasledovné účelové funkcie dvoch premenných bez ohraničení

- Kvadratická funkcia (konvexná kvadratická forma, jeden extrém)

$$J(x) = \frac{1}{2}x^T Ax + b^T x + c,\ \ A = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 3 \end{pmatrix}, b = \begin{pmatrix} -6 \\ 10 \end{pmatrix}, c = 10$$

Gradient:

$$\nabla J(x) = Ax + b$$

Hessián:

$$\nabla^2 J(x) = A$$

- Himmelblau funkcia (polynóm štvrtého stupňa, viacero extrémov)

$$J(x) = ({x_1}^2 + x_2 - 11)^2 + (x_1 + {x_2}^2 - 7)^2$$

Prvé derivácie:

$$\frac{\partial J(x)}{\partial x_1} = 4x_1({x_1}^2 + x_2 - 11) + 2(x_1 + {x_2}^2 - 7)$$

$$\frac{\partial J(x)}{\partial x_2} = 2({x_1}^2 + x_2 - 11) + 4x_2(x_1 + {x_2}^2 - 7)$$

Druhé derivácie:

$$\frac{\partial^2 J(x)}{\partial {x_1}^2} = 12{x_1}^2 + 4x_2 - 42$$

$$\frac{\partial^2 J(x)}{\partial {x_2}^2} = 2 + 4(x_1 + {x_2}^2 - 7) + 8{x_2}^2 = 12{x_2}^2 + 4x_1 - 26$$

$$\frac{\partial^2 J(x)}{\partial x_1 \partial x_2} = 4x_1 + 4x_2$$

Budeme aplikovať gradientné metódy optimalizácie bez ohraničení.

**Všeobecná gradientová metóda**

- V každom kroku sa vypočíta nový bod (aproximácia optima)
- Nový bod počítame rekurzívne ako

$$x(k+1) = x(k) - \gamma \nabla J\big(x(k)\big)$$

  kde $\nabla J\big(x(k)\big)$ je gradient v $x(k)$ a $\gamma > 0$ je relatívna veľkosť kroku

- Algoritmus ukončujeme ak je splnená podmienka (veľkosť gradientu)

$$\Big(\nabla J\big(x(k)\big)\Big)^T \nabla J\big(x(k)\big) < \epsilon$$

**Gradientová metóda s optimálnou dĺžkou kroku**

- Nový bod počítame rekurzívne ako

$$x(k+1) = x(k) - \gamma(k)\nabla J\big(x(k)\big)$$

- Optimálnu veľkosť kroku $\gamma(k)$ budeme voliť s využitím gradientu $\nabla J\big(x(k)\big)$ a Hessiánu $\nabla^2 J\big(x(k)\big)$ ako

$$\gamma(k) = \frac{\nabla J\big(x(k)\big)^T \nabla J\big(x(k)\big)}{\nabla J\big(x(k)\big)^T \nabla^2 J\big(x(k)\big) \nabla J\big(x(k)\big)}$$

- Algoritmus ukončujeme ak je splnená podmienka (veľkosť gradientu)

$$\Big(\nabla J\big(x(k)\big)\Big)^T \nabla J\big(x(k)\big) < \epsilon$$

**Newtonova-Raphsonova metóda**

- Nový bod počítame rekurzívne ako

$$x(k+1) = x(k) - \big[\nabla^2 J\big(x(k)\big)\big]^{-1} \nabla J\big(x(k)\big)$$

  kde $\big[\nabla^2 J\big(x(k)\big)\big]^{-1}$ je inverzná Hesseho matica

- Nie je nutné explicitne určiť inverznú Hesseho maticu. Postačuje vyriešiť sústavu lineárnych rovníc pre získanie kroku $x(k+1) - x(k)$

$$\nabla^2 J\big(x(k)\big)\big(x(k+1) - x(k)\big) = -\nabla J\big(x(k)\big)$$

- Algoritmus ukončujeme ak je splnená podmienka (veľkosť gradientu)

$$\Big(\nabla J\big(x(k)\big)\Big)^T \nabla J\big(x(k)\big) < \epsilon$$

## Úloha:

Použite skript *optim_scr*

- Analyzujte definitnosť Hessiánu v nájdenom extréme a určite o aký extrém sa jedná
- Analyzujte vplyv veľkosti kroku $\gamma$ na konvergenciu numerického riešenia a jeho stabilitu
- Analyzujte vplyv tolerancie $\epsilon$ na presnosť odhadu optima
- Analyzujte kolmosť smerov v prípade gradientovej metódy s optimálnou dĺžkou kroku
- Porovnajte rýchlosť konvergencie uvažovaných metód
- Analyzujte vplyv voľby počiatočného odhadu na priebeh a výsledok optimalizácie

Implementujte funkciu:

```
function [x_k, x_log, J_log, s_log] = optim_gradient(J_fun, x_0, eps, gamma, log)

x_log=[];
J_log=[];
s_log=[];
x_k=x_0;

while 1
```

```
    [J_k,g_xk,H] = J_fun(x_k);
    x_k=%TODO

    if log
        x_log=[x_log;(x_k+gamma*g_xk)'];
        J_log=[J_log;J_k];
        s_log=[s_log;-g_xk'];
    end

    if %TODO
        break;
    end

end
end
```

Implementujte funkciu:

```
function [x_k, x_log, J_log, s_log] = optim_gradient_optimal_step(J_fun, x_0, eps,
log)

x_log=[];
J_log=[];
s_log=[];
x_k=x_0;

while 1

    [J_k,g_xk,H] = J_fun(x_k);
    gamma=%TODO;
    x_k=%TODO;

    if log
        x_log=[x_log;(x_k+gamma*g_xk)'];
        J_log=[J_log;J_k];
        s_log=[s_log;-g_xk'];
    end

    if %TODO
        break;
    end

end
end
```

Implementujte funkciu:

```
function [x_k, x_log, J_log, s_log] = optim_newton_raphson(J_fun, x_k, eps, log)

x_log=[];
J_log=[];
s_log=[];
while 1

    [J_k,g_xk,H] = J_fun(x_k);
    x_k=%TODO

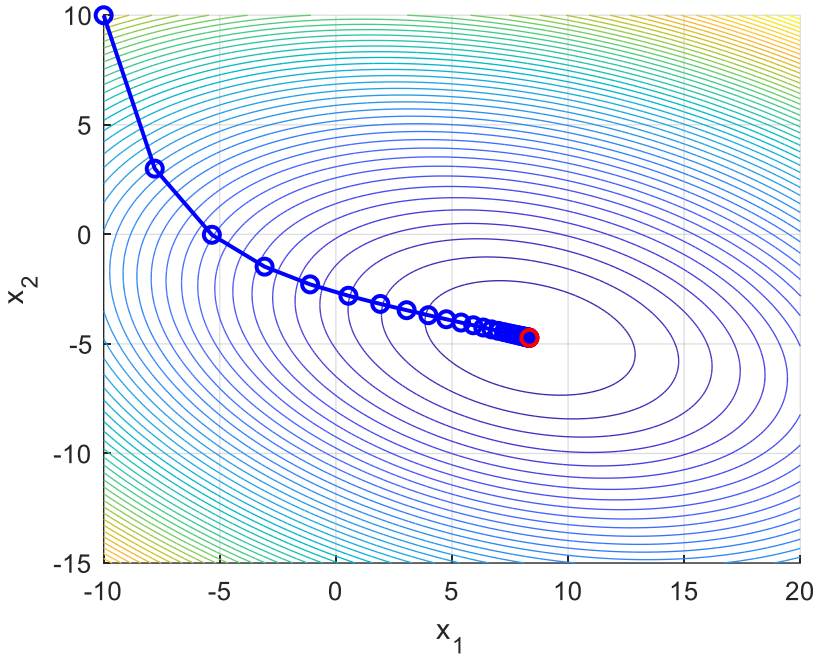
    if log
```

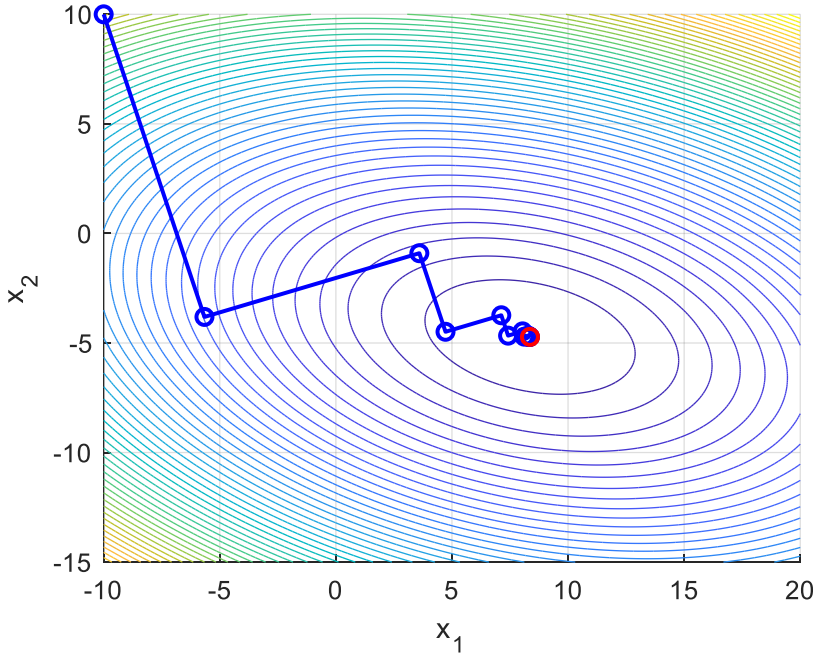
```
     x_log=[x_log;(x_k+H_k\g_k)'];
     J_log=[J_log;J_k];
     s_log=[s_log;(-H_k\g_k)'];
    end

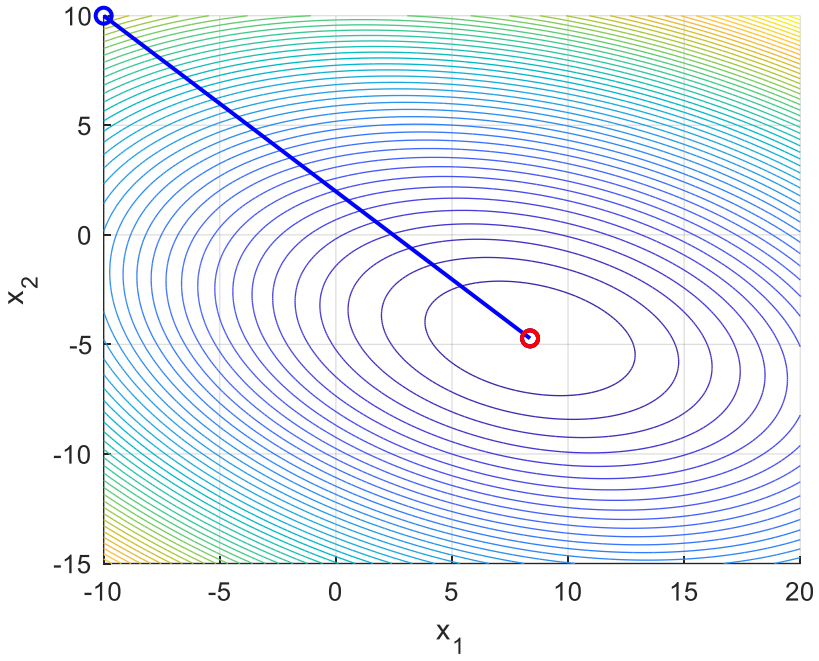
    if %TODO
        break;
    end
end

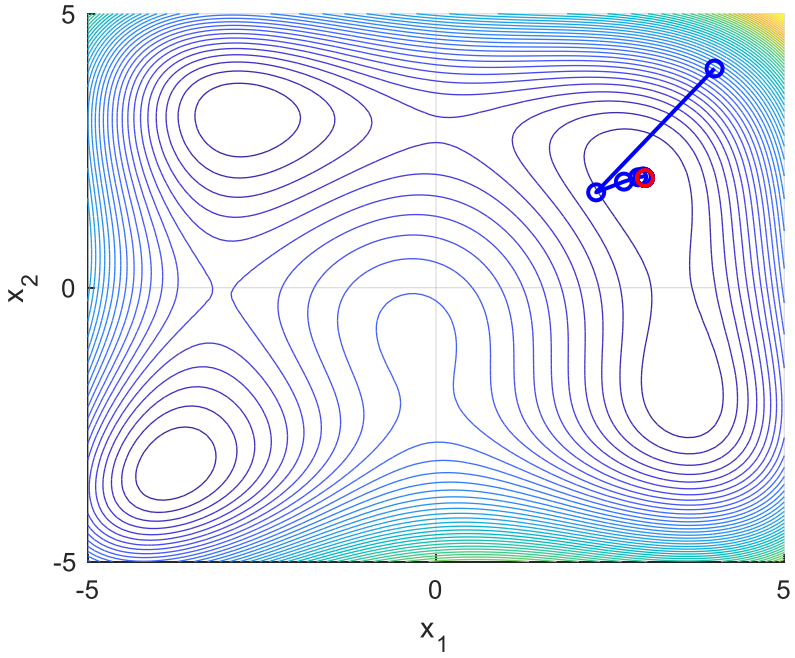
end
```

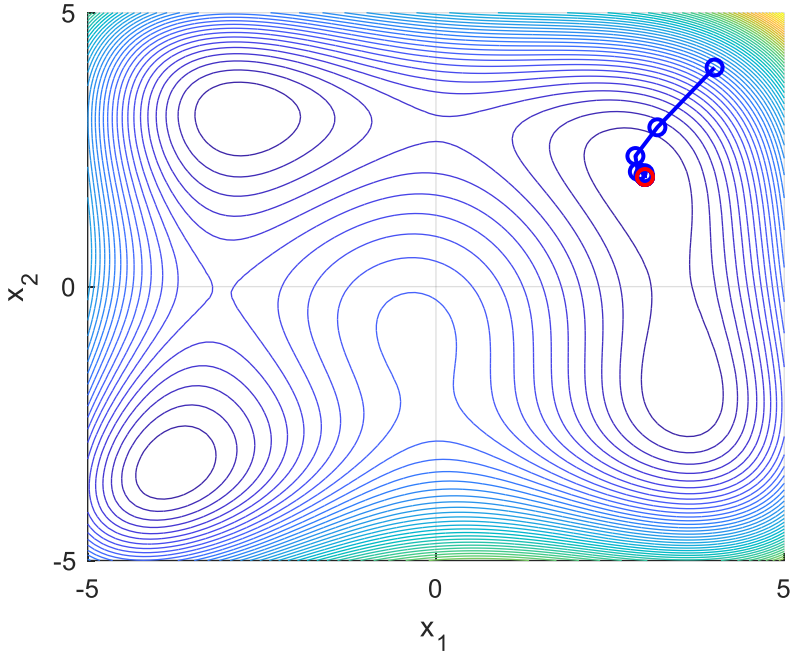
**Riešenie:**

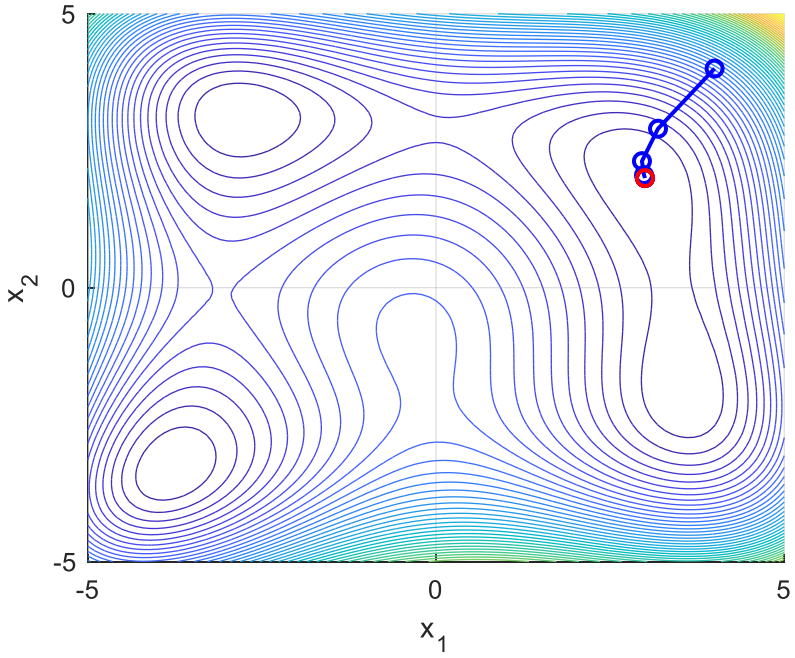



10
5
0
-5
-10
-15
-10
-5
0
5
10
15
20
x2
x1


5
0
-5
-5
0
5
x2
x1


5
0
-5
x2
-5
0
5
x1


5
0
-5
x2
-5
0
5
x1